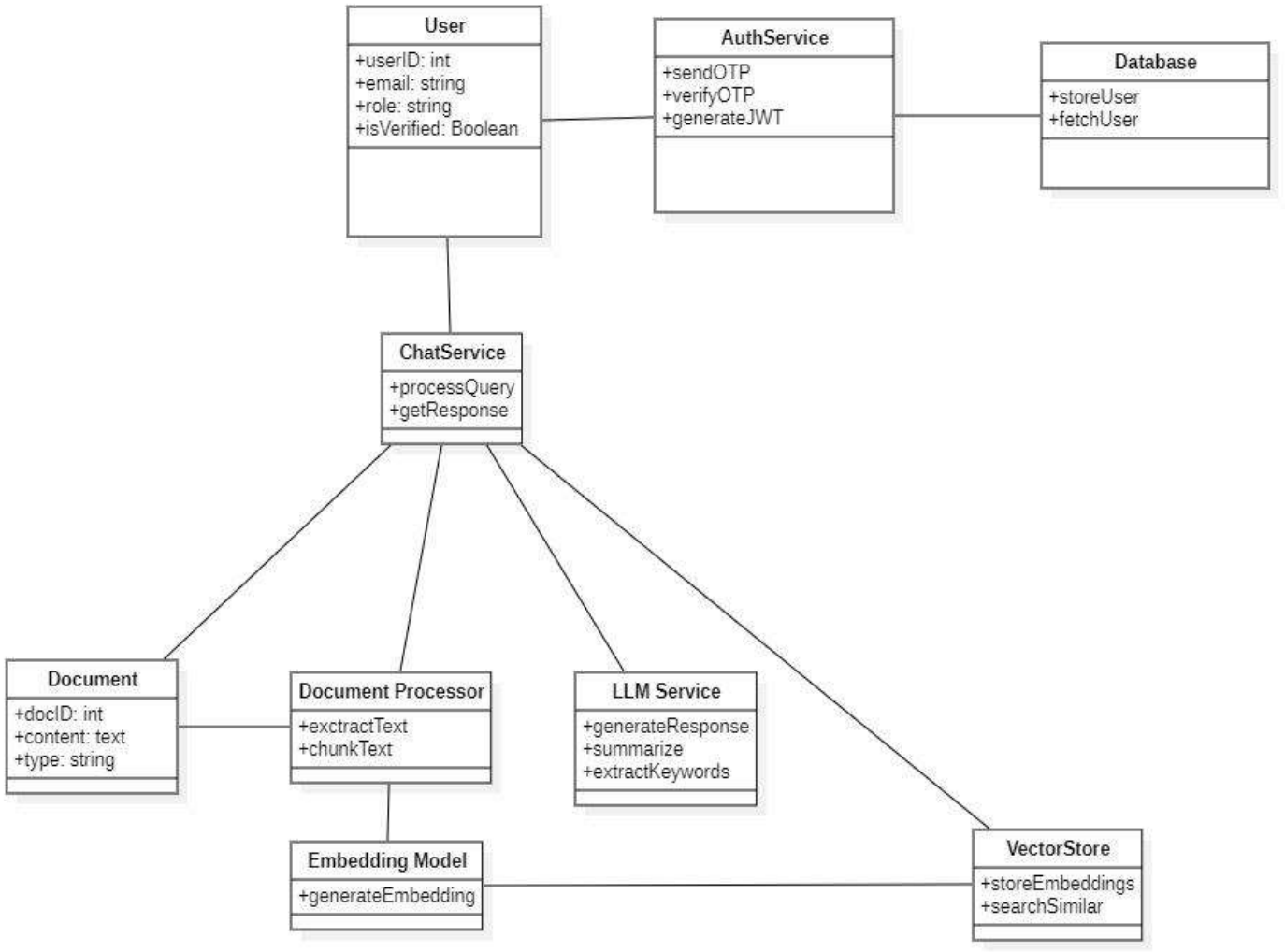
Class Diagram

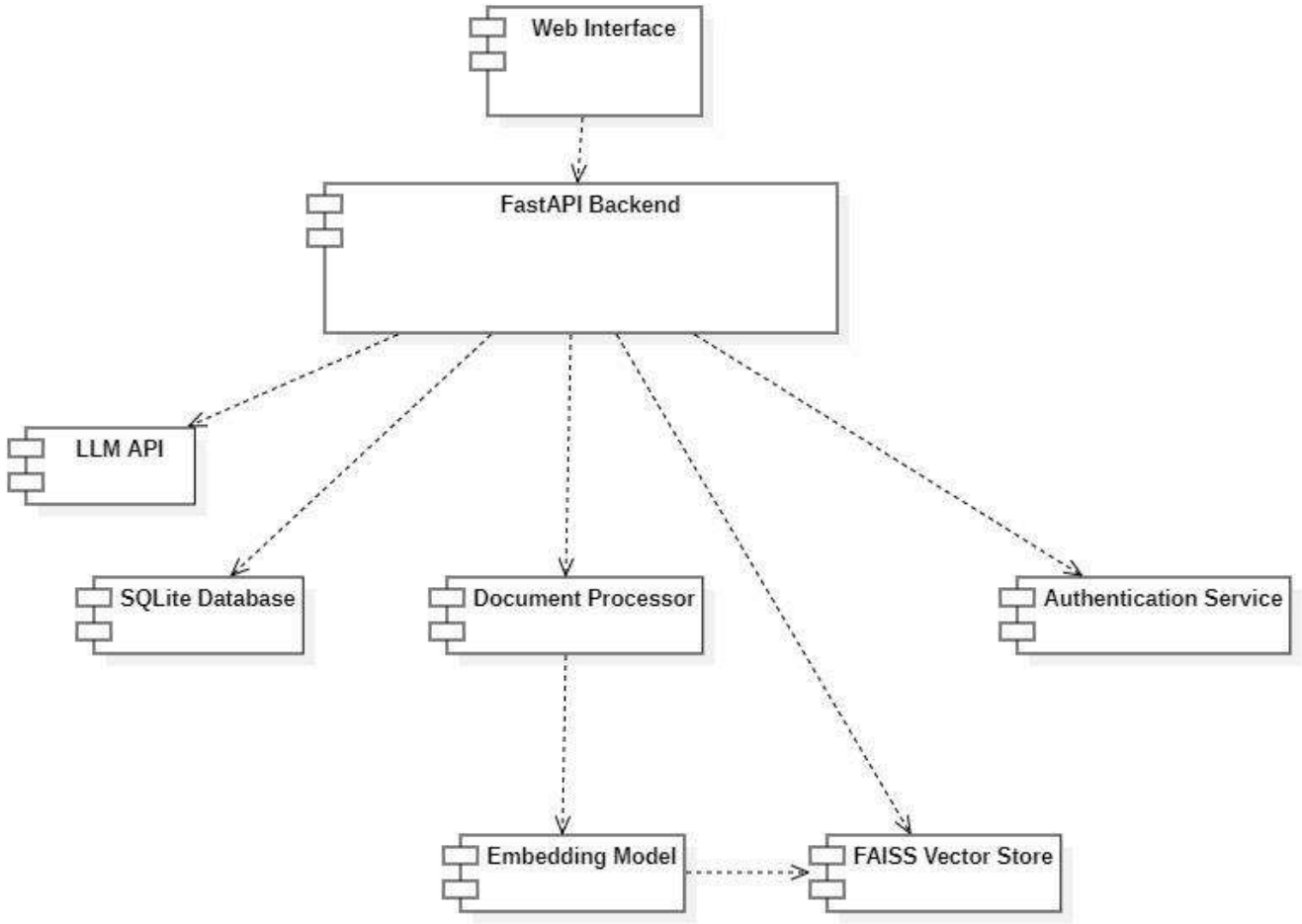


Description: The Class Diagram represents the static structure of the system by defining key classes, their attributes, and relationships. It includes classes such as:

- User
- AuthService
- ChatService
- DocumentProcessor
- EmbeddingModel
- VectorStore
- LLMService

The diagram captures how different components interact to implement authentication, document processing, and the RAG-based query mechanism. It also illustrates the separation of concerns across system modules, ensuring modularity and scalability.

Component Diagram



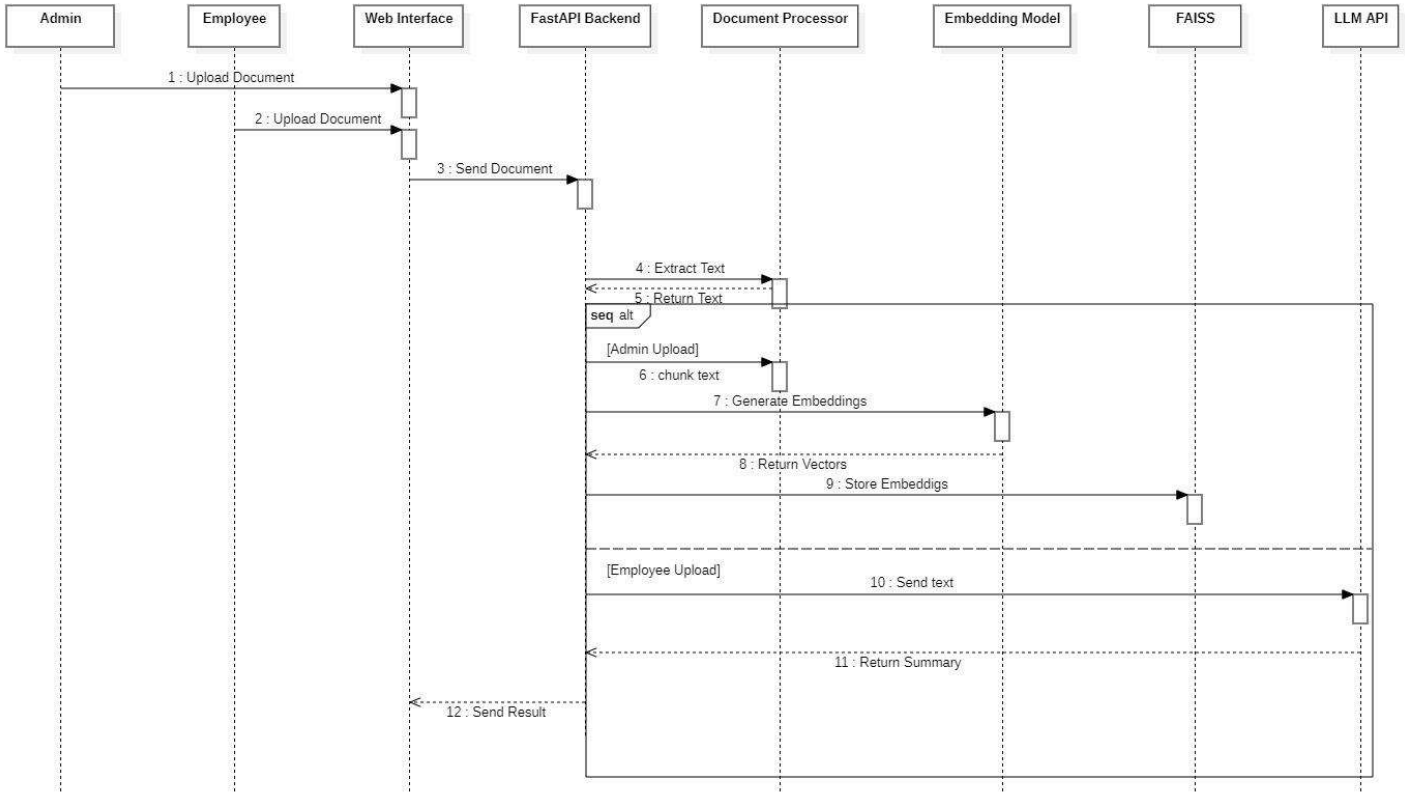
Description: The Component Diagram provides a high-level view of the system architecture by showing major components and their dependencies. It includes components such as:

- The Web Interface
- FastAPI Backend
- Authentication Service
- Document Processor
- Embedding Model
- FAISS Vector Store
- LLM API
- Database

The diagram demonstrates how the frontend interacts with the backend and how different services collaborate to perform authentication, document processing, and query handling. It reflects the modular design of the system.

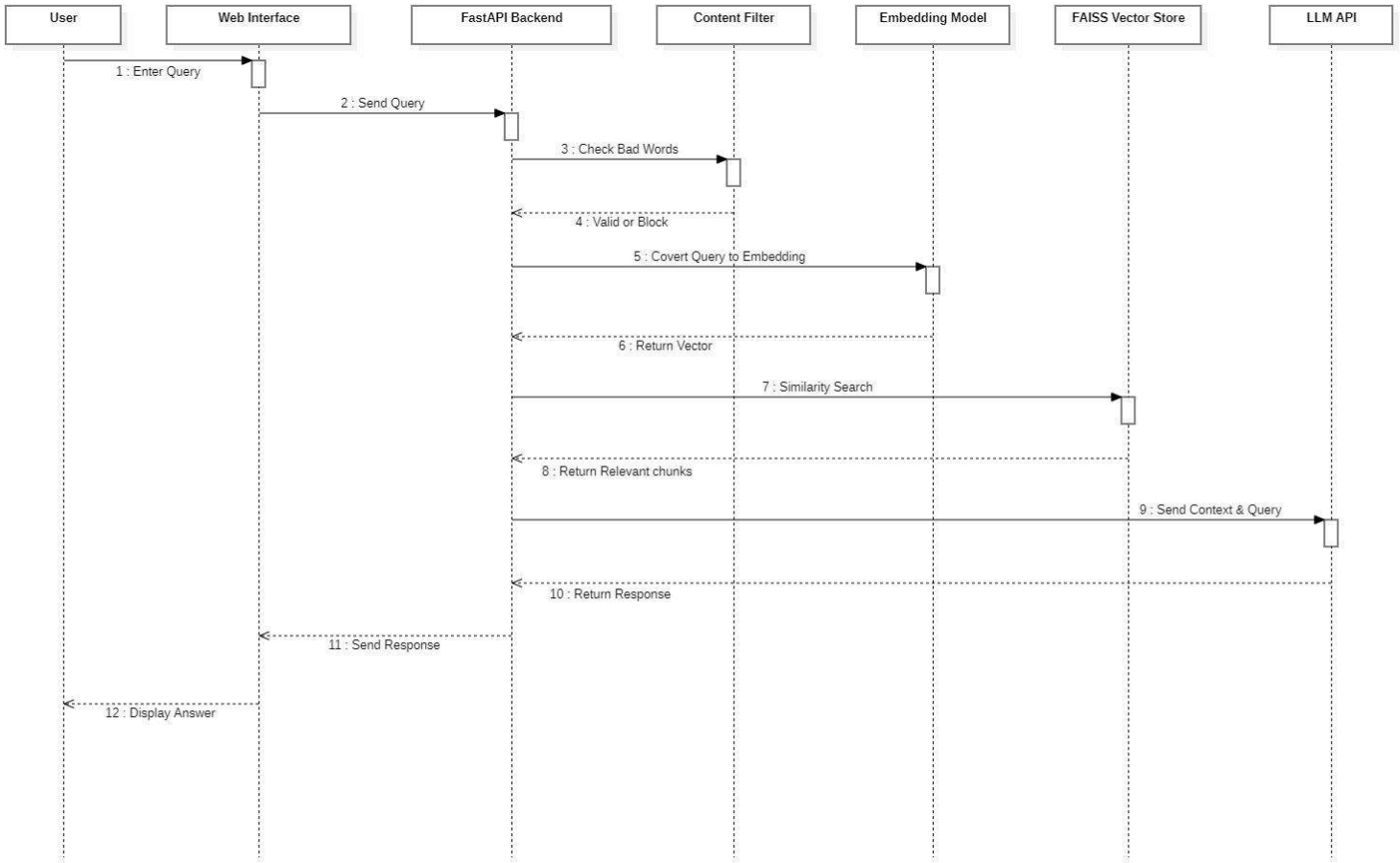
Sequence Diagram

Document Processing Sequence Diagram



Description: The Sequence Diagram for document processing illustrates two distinct workflows based on user roles. In the admin flow, uploaded policy documents are processed, chunked, converted into embeddings, and stored in the FAISS vector database for future retrieval. In the employee flow, uploaded personal documents are processed temporarily and sent directly to the LLM for summarization or keyword extraction without being stored. This diagram emphasizes role-based behavior and separation between persistent and non-persistent data processing.

Query Processing Sequence Diagram

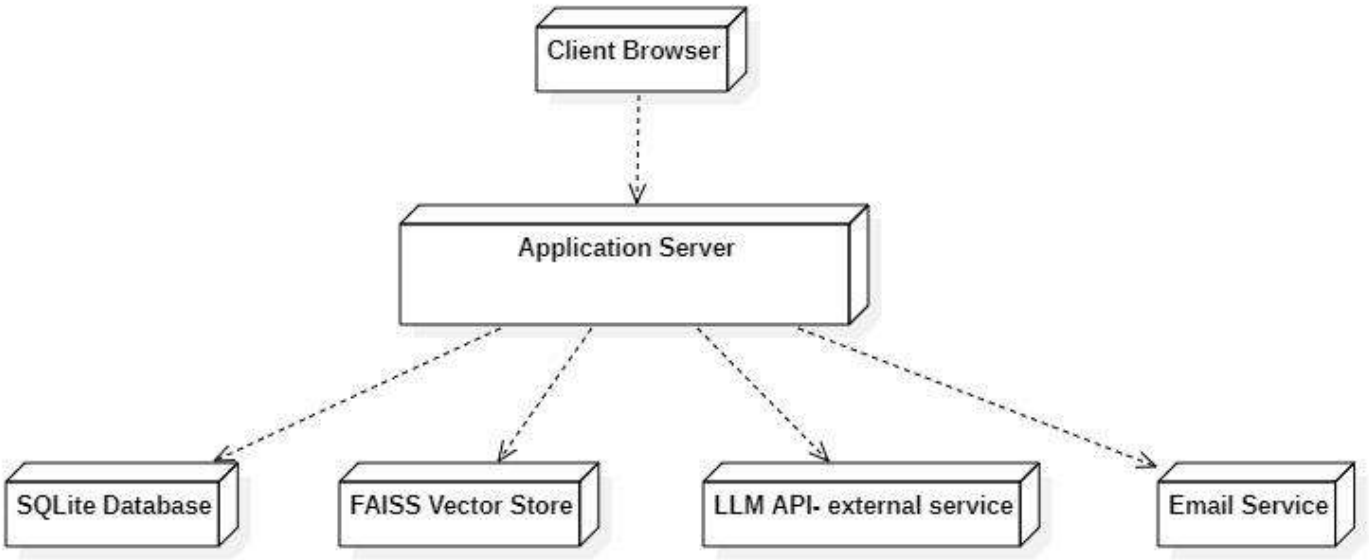


Description: The Sequence Diagram for document processing illustrates two distinct workflows based on user roles.

In the admin flow, uploaded policy documents are processed, chunked, converted into embeddings, and stored in the FAISS vector database for future retrieval.

In the employee flow, uploaded personal documents are processed temporarily and sent directly to the LLM for summarization or keyword extraction without being stored. This diagram emphasizes role-based behavior and separation between persistent and non-persistent data processing.

Deployment Diagram



Description: The Deployment Diagram illustrates the physical distribution of system components across different nodes.

The client interacts with the system through a web browser, which communicates with the application server hosting the FastAPI backend.

The backend integrates multiple services including the FAISS vector store for semantic retrieval, SQLite database for user data, an external LLM API for response generation, and an SMTP-based email service for authentication. This architecture reflects a scalable client-server model where core processing is handled centrally while leveraging external AI services.

4. Current Status

The project has successfully completed the requirement analysis and system design phases, including the development of UML diagrams such as Use Case, Activity, Sequence, Class, Component, and Deployment diagrams. The system architecture has been finalized, and the RAG-based workflow has been clearly defined. The project is now ready to proceed to the implementation phase.

5. Future Work

The next phase of the project will focus on implementing the core system functionalities, including

- Backend API development using FastAPI
- Integration of FAISS for vector-based retrieval and incorporation of an LLM API for response generation.
- Implementing authentication using email OTP
- Developing the user interface
- Conducting testing for performance and scalability and deploying the system on a cloud platform.